

Custodial Crypto Wallet

Проектная работа по курсу Базы Данных



Защита проекта на тему: Кастодиальный крипто- кошелек

Обо мне:
Хлименков Артем
Senior Backend Engineer

Мой стек:
[Node.js](#), RabbitMQ, PostgreSQL,
Redis, Blockchain/FinTech/DeFi



План защиты



- Цель и задачи проекта
- Какие технологии использовались
- Что получилось в итоге
- Выводы
- Вопросы и рекомендации

Цель и задачи проекта

- **Цель:** спроектировать БД для кастодиального крипто-кошелька.
Что такое кастодиальный кошелек? Кастодиальный кошелек — это вид кошелька для криптовалют, где приватные ключи и сами транзакции хранятся у третьей стороны — кастодиала по типу централизованной биржи.
- **Задачи:**
 - хранение данных о мерчантах их кошельках и клиентах;
 - приём криптовалютных платежей от клиентов мерчантов через персональные кошельки и пополнения самих мерчантов через транзитные кошельки;
 - выплаты с платформы на внешние blockchain-адреса;
 - возвраты (refunds) средств по транзакциям не прошедших AML (Anti-Money Laundering) проверки;
 - AML-проверки входящих транзакций через сторонний сервис;
 - учет движения средств на аккаунтах с детализацией комиссий;
 - хранение кастомных тарифов для мерчантов;
 - callback-уведомления мерчантов о статусах транзакций;
 - агрегация данных для клиента BO (бек-офис) с помощью обобщающей таблицы;

Какие технологии использовались?

- PostgreSQL 14+ – основная СУБД проекта
- SQL – схема, ограничения, хранимые процедуры, триггеры, выборки данных
- Расширение: uuid-oss (генерация UUID v4)
- ORM в production: TypeORM (Node.js)
- Типы данных: ENUM, UUID, JSONB, CIDR[], numeric (финансовые расчёты без потерь точности)
- Docker/Docker-compose – сборка контейнера с БД
- DBeaver – основной инструмент для работы с БД



Структура репозитория

custodial-crypto-wallet/



├── DDL/

- ├── create_db.sql – создание БД, ролей и пользователей
- ├── create_tables.sql – создание всех таблиц и enum-типов
- ├── create_indexes.sql – создание индексов
- ├── stored_procedures.sql – хранимые процедуры
- └── triggers.sql – триггеры

└── DML/

- ├── insert_data.sql – примеры вставки данных
- ├── update_data.sql – примеры обновления данных
- └── select_data.sql – примеры выборок

Запуск через Docker

Требования

- [Docker](#) 24+
- [Docker Compose](#) v2+

Запуск базы данных

```
# Запустить PostgreSQL в фоне (схема создаётся автоматически)
docker compose up -d

# Проверить статус (ждать healthy)
docker compose ps

# Просмотр логов инициализации
docker compose logs postgres
```

После старта база доступна по адресу `localhost:5433` .

Скрипты `DDL/create_tables.sql`, `create_indexes.sql`, `stored_procedures.sql` и `triggers.sql` применяются автоматически при первом запуске (через `docker-entrypoint-initdb.d`).

Остановка и очистка

```
# Остановить контейнер (данные сохраняются в volume)
docker compose down

# Остановить и удалить данные (полный сброс)
docker compose down -v
```

Загрузка тестовых данных вручную

```
# Подключиться к контейнеру и выполнить DML
docker exec -i custodial_crypto_wallet_db psql -U payment_admin -d custodial_crypto_wallet_db <
```

Ссылка на схему базы данных:

https://github.com/brizenorton/custodial-crypto-wallet/blob/main/custodial_crypto_wallet_db%20-%20custodial_crypto_wallet_db%20-%20public.png

Схема достаточно большая, поэтому лучше посмотреть ее сразу в репозитории.

Ссылка на репозиторий проекта: <https://github.com/brizenorton/custodial-crypto-wallet/tree/main>

Этапы проектирования



Create_db.sql пример

-- Создание базы данных

```
CREATE DATABASE custodial_crypto_wallet_db;
```

-- Подключиться к БД перед выполнением остальных команд:

```
-- \c custodial_crypto_wallet_db
```

-- Создание схемы (таблицы используют схему public по умолчанию)

```
CREATE SCHEMA IF NOT EXISTS public;
```

-- Роль администратора: полный доступ

```
CREATE ROLE admin_role;
```

```
GRANT ALL PRIVILEGES ON DATABASE custodial_crypto_wallet_db TO admin_role;
```

```
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_role;
```

```
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO admin_role;
```

```
GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public TO admin_role;
```

-- Чтобы привилегии распространялись на новые таблицы:

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL PRIVILEGES ON TABLES TO admin_role;
```

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL PRIVILEGES ON SEQUENCES TO admin_role;
```

Create_tables.sql (enums)

```
CREATE TYPE "public"."CryptoCurrency" AS ENUM (  
    'BTC', 'LTC', 'BNB', 'TRX', 'ETH', 'TON',  
    'USDT-BEP20', 'USDT-TRC20', 'USDT-ERC20', 'USDT-TEP74',  
    'USDT-ARB-ERC20', 'USDT-AVAX-ERC20', 'USDT-BASE-ERC20', 'USDT-OP-ERC20', 'USDT-POL-ERC20', 'USDT-SOL',  
    'USDC-ERC20', 'USDC-BEP20', 'USDC-ARB-ERC20', 'USDC-AVAX-ERC20', 'USDC-BASE-ERC20', 'USDC-OP-ERC20', 'USDC-POL-ERC20', 'USDC-SOL',  
    'ETH-ARB', 'ETH-BASE', 'ETH-BEP20', 'ETH-OP',  
    'ARB-ERC20', 'AVAX', 'SOL', 'POL', 'BCH', 'DOGE', 'XRP', 'ADA', 'ALGO',  
    'WBTC-ERC20', 'WBTC-ARB-ERC20', 'WBTC-BASE-ERC20', 'WBTC-OP-ERC20', 'WBTC-POL-ERC20',  
    'WETH-ERC20', 'WETH-ARB-ERC20', 'WETH-BASE-ERC20', 'WETH-BEP20', 'WETH-POL-ERC20',  
    'LINK-ERC20', 'LINK-BEP20', 'SHIB-ERC20', 'SHIB-BEP20',  
    'UNI-ERC20', 'UNI-BEP20', 'DAI-ERC20', 'DAI-BEP20',  
    'BNB-ERC20', 'BNB-POL-ERC20', 'BTCB-BEP20', 'DOGE-BEP20',  
    'POL-ERC20', 'POL-BEP20', 'SOL-BEP20', 'XRP-BEP20',  
    'WBNB-POL-ERC20', 'WSOL-POL-ERC20', 'WXP-ERC20'  
);  
  
CREATE TYPE "public"."RateCryptoCurrency" AS ENUM (  
    'BNB', 'BTC', 'ETH', 'LTC', 'TRX', 'TON', 'USDT', 'USDC',  
    'ARB', 'AVAX', 'SOL', 'POL', 'BCH', 'ADA', 'ALGO', 'LINK', 'SHIB', 'UNI',  
    'DOGE', 'XRP'  
);  
  
CREATE TYPE "public"."FiatCurrency" AS ENUM (  
    'USD', 'EUR', 'RUB', 'UAH', 'KZT', 'AZN', 'BDT', 'BGN', 'BRL',  
    'BYN', 'CAD', 'AUD', 'HUF', 'INR', 'JPY', 'KRW', 'MXN',  
    'NOK', 'PLN', 'RON', 'SEK', 'TRY', 'UZS'  
);
```

Create_tables.sql (tables)

```

-----
-- accounts
-- Базовая таблица-идентификатор аккаунта.
-- К аккаунту привязаны мерчанты, кошельки и транзакции.
-----
CREATE TABLE IF NOT EXISTS "accounts" (
    "id"          uuid          NOT NULL DEFAULT uuid_generate_v4(),
    "created_at"  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    "updated_at"  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT "PK_accounts" PRIMARY KEY ("id")
);

-----
-- rates
-- Курсы криптовалют к фиатным валютам.
-- Хранит актуальные и исторические курсы; fiat_rates — JSON-объект вида {"USD":1.5,...}
-----
CREATE TABLE IF NOT EXISTS "rates" (
    "id"          uuid          NOT NULL DEFAULT uuid_generate_v4(),
    "created_at"  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    "updated_at"  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    "fiat_rates"  jsonb,
    "crypto_currency" "public"."RateCryptoCurrency" NOT NULL,
    "type"         "public"."RateType"             NOT NULL DEFAULT 'EXTERNAL',
    CONSTRAINT "PK_rates" PRIMARY KEY ("id")
);

```

```

-----
-- merchants
-- Мерчант — юридическое лицо, использующее платёжную систему.
-- Привязан к аккаунту (account_id) для учёта баланса.
-----
CREATE TABLE IF NOT EXISTS "merchants" (
    "id"          uuid          NOT NULL DEFAULT uuid_generate_v4(),
    "created_at"  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    "updated_at"  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    "name"        character varying NOT NULL,
    "public_key"  character varying NOT NULL,
    "private_key" character varying NOT NULL,
    "encrypted_public_key" character varying,
    "encrypted_private_key" character varying NOT NULL,
    "account_id"  uuid          UNIQUE,
    "type"        "public"."MerchantType" NOT NULL DEFAULT 'ENTERPRISE',
    CONSTRAINT "PK_merchants" PRIMARY KEY ("id"),
    CONSTRAINT "FK_merchants_accounts" FOREIGN KEY ("account_id")
        REFERENCES "accounts" ("id") ON DELETE NO ACTION ON UPDATE NO ACTION
);

```

Create_indexes.sql

```
-- Основной индекс для поиска последнего курса по валюте и типу
CREATE INDEX IF NOT EXISTS "Rate_IDX_cryptoCurrency_type_createdAt_DESC"
  ON "rates" ("crypto_currency", "type", "created_at" DESC);

-- Индекс для поиска последнего курса только по типу
CREATE INDEX IF NOT EXISTS "Rate_IDX_type_createdAt_DESC"
  ON "rates" ("type", "created_at" DESC);

-- Индекс для задачи очистки старых курсов (cleanup job)
CREATE INDEX IF NOT EXISTS "Rate_IDX_createdAt"
  ON "rates" ("created_at");

-- Составной индекс для очистки по дате (используется в batch-delete)
CREATE INDEX IF NOT EXISTS "Rate_IDX_cryptoCurrency_type_Date_createdAt_DESC"
  ON "rates" ("crypto_currency", "type", DATE("created_at"), "created_at" DESC);

-- Индекс для поиска курсов по объекту фиатных валют fiat_rates
CREATE INDEX IF NOT EXISTS "Rate_IDX_fiatRates" ON "rates" USING GIN ("fiat_rates");
```

```
-- Составной индекс для поиска выплат по мерчанту и валюте
```

```
CREATE INDEX "Payout_IDX_merchantId_currency"
  ON "payouts" ("merchant_id", "crypto_currency");
```

```
-- Индекс для FK на rates (cleanup rates job)
```

```
CREATE INDEX IF NOT EXISTS "Payout_IDX_rateId"
  ON "payouts" ("rate_id");
```

```
-- BRIN-индекс для партиционирования по дате создания (большой объем данных)
```

```
CREATE INDEX IF NOT EXISTS "Payout_IDX_createdAt_brin"
  ON "payouts" USING BRIN ("created_at");
```

Stored_procedures.sql / Triggers.sql

```

CREATE OR REPLACE PROCEDURE create_merchant_wallet
OUT p_error_code INT, -- Возвращает в начало
p_merchant_id UUID,
p_address VARCHAR(255),
p_crypto_currency VARCHAR(64),
p_account_id UUID DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_wallet_id UUID;
BEGIN
    p_error_code := 0;

    IF NOT EXISTS (SELECT 1 FROM merchants WHERE id = p_merchant_id) THEN
        p_error_code := 300;
        RAISE NOTICE 'Merchant with id % does not exist', p_merchant_id;
        RETURN;
    END IF;

    IF p_account_id IS NOT NULL AND NOT EXISTS (SELECT 1 FROM accounts WHERE id = p_account_id) THEN
        p_error_code := 300;
        RAISE NOTICE 'Account with id % does not exist', p_account_id;
        RETURN;
    END IF;

    IF EXISTS (
        SELECT 1 FROM merchant_wallets
        WHERE address = p_address
        AND crypto_currency = p_crypto_currency::"public"."CryptoCurrency"
    ) THEN
        p_error_code := 100;
        RAISE NOTICE 'Wallet with address % and currency % already exists', p_address, p_crypto_currency;
        RETURN;
    END IF;

    INSERT INTO merchant_wallets
    (merchant_id, address, crypto_currency, account_id)
    VALUES
    (p_merchant_id, p_address, p_crypto_currency::"public"."CryptoCurrency", p_account_id)
    RETURNING id INTO v_wallet_id;

    RAISE NOTICE 'Merchant wallet created with id: %', v_wallet_id;

EXCEPTION
    WHEN NOT_NULL_VIOLATION OR FOREIGN_KEY_VIOLATION OR UNIQUE_VIOLATION THEN
        p_error_code := 100;
        RAISE NOTICE 'Error creating merchant wallet: constraint violation - %', SQLERRM;
    WHEN OTHERS THEN
        p_error_code := 200;
        RAISE NOTICE 'Error creating merchant wallet: %', SQLERRM;

END;
$$;

```

```

-- Функция: trigger_mtw_to_callback_history
-- Назначение: При появлении новой транзакции транзитного кошелька
-- создаёт начальную запись в callback_histories.

CREATE OR REPLACE FUNCTION trigger_mtw_to_callback_history()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO callback_histories
    (type, url, type_external_id, sent_data, payload_status)
    VALUES (
        'MERCHANT_TRANSIT_WALLET_TRANSACTION',
        '', -- у mtwt нет прямого callback_url в таблице; будет заполнено сервисом
        NEW.id::text,
        jsonb_build_object(
            'id', NEW.id,
            'status', NEW.status,
            'crypto_currency', NEW.crypto_currency,
            'crypto_value', NEW.crypto_value,
            'fee', NEW.fee,
            'to', NEW.to,
            'merchant_id', NEW.merchant_id,
            'type', NEW.type,
            'created_at', NEW.created_at
        ),
        'PENDING'::"public"."CallbackPayloadStatus"
    );

    RETURN NEW;
END;
$$;

```

insert_data.sql

```
-- =====
-- rates
-- Курсы криптовалют; fiat_rates – JSON с котировками к фиатным валютам
-- =====

INSERT INTO rates (id, fiat_rates, crypto_currency, type)
VALUES
(
    'b2000000-0000-0000-0000-000000000001',
    '{"USD": 67450.50, "EUR": 62300.00, "RUB": 620000.00}':::jsonb,
    'BTC':::"public", "RateCryptoCurrency",
    'EXTERNAL':::"public", "RateType"
),
(
    'b2000000-0000-0000-0000-000000000002',
    '{"USD": 3520.75, "EUR": 3248.00, "RUB": 323500.00}':::jsonb,
    'ETH':::"public", "RateCryptoCurrency",
    'EXTERNAL':::"public", "RateType"
),
(
    'b2000000-0000-0000-0000-000000000003',
    '{"USD": 1.001, "EUR": 0.923, "RUB": 92.00}':::jsonb,
    'USDT':::"public", "RateCryptoCurrency",
    'EXTERNAL':::"public", "RateType"
),
(
    'b2000000-0000-0000-0000-000000000004',
    '{"USD": 0.1285, "EUR": 0.1186, "RUB": 11.82}':::jsonb,
    'TRX':::"public", "RateCryptoCurrency",
    'EXTERNAL':::"public", "RateType"
)
ON CONFLICT (id) DO NOTHING;
```

```
-- =====
-- merchants
-- Мерчанты; привязаны к аккаунту через account_id
-- =====

INSERT INTO merchants (id, name, public_key, private_key, encrypted_private_key, type, account_id)
VALUES
(
    'c3000000-0000-0000-0000-000000000001',
    'Acme Corp',
    'pk_acme_pub_key_example',
    'pk_acme_priv_key_example',
    'encrypted_priv_key_acme',
    'ENTERPRISE':::"public", "MerchantType",
    'a1000000-0000-0000-0000-000000000001'
),
(
    'c3000000-0000-0000-0000-000000000002',
    'Small Shop LLC',
    'pk_shop_pub_key_example',
    'pk_shop_priv_key_example',
    'encrypted_priv_key_shop',
    'SMB':::"public", "MerchantType",
    'a1000000-0000-0000-0000-000000000002'
)
ON CONFLICT (id) DO NOTHING;
```

select_data.sql

```

-- Получить последние курсы для всех криптовалют (DISTINCT ON – PostgreSQL)
SELECT DISTINCT ON (crypto_currency)
    id, crypto_currency, fiat_rates, type, created_at
FROM rates
WHERE type = 'EXTERNAL'::"public"."RateType"
ORDER BY crypto_currency, created_at DESC;

-- Рекурсивный CTE: последний курс по каждой валюте (производительный вариант)
WITH RECURSIVE cte AS (
    (
        SELECT id, crypto_currency, fiat_rates, type, created_at
        FROM rates
        WHERE type = 'EXTERNAL'::"public"."RateType"
        ORDER BY crypto_currency, created_at DESC
        LIMIT 1
    )
    UNION ALL
    SELECT r.*
    FROM cte c
    CROSS JOIN LATERAL (
        SELECT r.id, r.crypto_currency, r.fiat_rates, r.type, r.created_at
        FROM rates r
        WHERE r.type = 'EXTERNAL'::"public"."RateType"
        AND r.crypto_currency > c.crypto_currency
        ORDER BY r.crypto_currency, r.created_at DESC
        LIMIT 1
    ) r
)
TABLE cte ORDER BY crypto_currency;

```

```

-- Получить мерчанта по ID с его кошельками
SELECT m.id, m.name, m.type, m.account_id,
    w.id AS wallet_id, w.address, w.crypto_currency
FROM merchants m
LEFT JOIN merchant_wallets w ON m.id = w.merchant_id
WHERE m.id = 'c3000000-0000-0000-0000-000000000001';

-- Получить мерчанта по account_id
SELECT m.id, m.name, m.type, m.account_id
FROM merchants m
WHERE m.account_id = 'a1000000-0000-0000-0000-000000000001';

-- Список всех мерчантов с типом ENTERPRISE
SELECT id, name, type, created_at
FROM merchants
WHERE type = 'ENTERPRISE'::"public"."MerchantType"
ORDER BY created_at DESC;

-- Поиск мерчантов по массиву ID (для bulk-операций)
SELECT id, name, account_id
FROM merchants
WHERE id = ANY(
    ARRAY[
        'c3000000-0000-0000-0000-000000000001',
        'c3000000-0000-0000-0000-000000000002'
    ]::uuid[]
);

```


update_data.sql

```
-- Депрекация кошелька клиента (например, при смене адреса)
UPDATE customer_wallets
SET status      = 'DEPRECATED':"public"."WalletStatus",
    updated_at = now()
WHERE id = 'f6000000-0000-0000-0000-000000000002';

-- Привязка email клиента к кошельку
UPDATE customer_wallets
SET user_email = 'client-002@example.com',
    updated_at = now()
WHERE merchant_id = 'c3000000-0000-0000-0000-000000000001'
    AND customer_id = 'client-002';

-- Обновление callback URL для всех кошельков мерчанта
UPDATE customer_wallets
SET callback_url = 'https://acme-new.example.com/callbacks/wallet',
    updated_at   = now()
WHERE merchant_id = 'c3000000-0000-0000-0000-000000000001';
```

```
-- Удаление старых записей в конфигурируемом промежутке, оставляя последние на каждый день.
-- Запускается через планировщик в коде
WITH day_ends AS (
    SELECT DISTINCT ON ("crypto_currency", "type", DATE("created_at"))
        "id"
    FROM "rates"
    ORDER BY "crypto_currency", "type", DATE("created_at"), "created_at" DESC
)
DELETE FROM rates WHERE id IN (
    SELECT r1.id
    FROM rates r1
    WHERE r1.created_at <= NOW() - make_interval(hours => 1)
    AND r1.created_at >= NOW() - make_interval(hours => 12)
    -- Preserve end-of-day snapshot (latest rate per currency/type/day)
    AND r1.id NOT IN (SELECT id FROM day_ends)
    -- Do not delete if referenced in other tables
    AND NOT EXISTS (SELECT 1 FROM payouts p WHERE p.rate_id = r1.id)
    AND NOT EXISTS (SELECT 1 FROM customer_wallet_transactions cwt WHERE cwt.rate_id = r1.id)
    AND NOT EXISTS (SELECT 1 FROM merchant_transit_wallet_transactions mtwt WHERE mtwt.rate_id = r1.id)
    AND NOT EXISTS (SELECT 1 FROM transaction_data td WHERE td.rate_id = r1.id)
    LIMIT 5000;
)
RETURNING id;
```

Выводы:

Был создан учебный проект базы данных custodial-crypto-wallet – полностью документированная, запускаемая через Docker реляционная БД PostgreSQL.

1. Нормализация vs. денормализация применяется осознанно.

Большинство таблиц нормализованы (3NF), но `transaction_data` — намеренно денормализована: она агрегирует данные из разных источников в одну таблицу для ВО и отчётности, жертвуя избыточностью ради скорости чтения.

2. Баланс — виртуальная величина, не хранится явно.

Таблица `accounts` не имеет поля `balance`. Баланс вычисляется динамически из трёх источников: `account_transactions` (движение средств) — `account_transaction_fees` (комиссии) — `account_fees` (legacy-списания). Это исключает рассинхронизацию и гарантирует аудируемость каждой копейки.

3. Идемпотентность реализована на уровне БД.

`payouts`: UNIQUE по `(external_id, merchant_id)`. `transaction_data`: ON CONFLICT DO UPDATE. `account_transactions`: UNIQUE INDEX по `(original_transaction_id, type)`. Повторный INSERT не дублирует запись — это защита от сетевых ретраев без бизнес-логики в приложении.

4. Индексы решают конкретные задачи, не создаются "про запас".

Partial-индексы (`WHERE is_collected = false`, `WHERE txid IS NOT NULL`) сокращают размер индекса и ускоряют именно горячие запросы. BRIN-индекс на `payouts.created_at` эффективен для монотонно растущих временных рядов с большим объёмом данных.

5. Триггеры реализуют принцип «один источник правды».

Вместо того чтобы каждый сервис приложения вручную писал в `callback_histories` и `account_transactions`, триггеры гарантируют создание этих записей при любом INSERT в родительские таблицы — даже при прямых SQL-операциях в обход ORM.

6. Ролевая модель разделяет доступ по назначению, а не по принципу минимализма.

`developer_role`, `qa_role` и `devops_role` имеют одинаковые технические права, но разделены для аудита и будущего точечного ограничения. `analyst_role` — только SELECT, без возможности изменить данные.

Выводы:

1. Нормализация vs денормализация применяется осознанно.

Большинство таблиц нормализованы (3NF), но `transaction_data` — намеренно денормализована: она агрегирует данные из разных источников в одну таблицу для ВО и отчётности, жертвуя избыточностью ради скорости чтения.

2. Баланс — виртуальная величина, не хранится явно.

Таблица `accounts` не имеет поля `balance`. Баланс вычисляется динамически из трёх источников: `account_transactions` (движение средств) – `account_transaction_fees` (комиссии) – `account_fees` (legacy-списания). Это исключает рассинхронизацию и гарантирует аудируемость каждой копейки.

3. Идемпотентность реализована на уровне БД. `payouts`: UNIQUE по (`external_id`, `merchant_id`). `transaction_data`: ON CONFLICT DO UPDATE. `account_transactions`: UNIQUE INDEX по (`original_transaction_id`, `type`). Повторный INSERT не дублирует запись — это защита от сетевых ретраев без бизнес-логики в приложении.

4. Индексы решают конкретные задачи, не создаются "про запас".

Partial-индексы (`WHERE is_collected = false`, `WHERE txid IS NOT NULL`) сокращают размер индекса и ускоряют именно горячие запросы. BRIN-индекс на `payouts.created_at` эффективен для монотонно растущих временных рядов с большим объёмом данных.

5. Триггеры реализуют принцип «один источник правды».

Вместо того чтобы каждый сервис приложения вручную писал в `callback_histories` и `account_transactions`, триггеры гарантируют создание этих записей при любом INSERT в родительские таблицы — даже при прямых SQL-операциях в обход ORM.

6. Ролевая модель разделяет доступ по назначению, а не по принципу минимализма. `developer_role`, `qa_role` и `devops_role` имеют одинаковые технические права, но разделены для аудита и будущего точечного ограничения. `analyst_role` — только SELECT, без возможности изменить данные.

Выводы:

По итогу курса был создан рабочий проект базы данных custodial-crypto-wallet – полностью документированная, запускаемая через Docker реляционная БД PostgreSQL. Все цели выполнены, сложностей в процессе изучения курса не возникало. Проект оцениваю на 8/10. По времени заняло примерно 4 дня.

DDL — Определение структуры данных

Файл	Содержание
<code>create_db.sql</code>	База данных <code>custodial_crypto_wallet_db</code> , 5 ролей, 5 пользователей
<code>create_tables.sql</code>	32 enum-типа, 22 таблицы с FK, UNIQUE и CHECK-ограничениями
<code>create_indexes.sql</code>	~35 индексов: B-tree, partial, BRIN, CONCURRENTLY
<code>stored_procedures.sql</code>	14 хранимых процедур с OUT-кодами ошибок 0 / 100 / 200 / 300 / 400
<code>triggers.sql</code>	6 триггеров AFTER INSERT → агрегация в <code>callback_histories</code> и <code>account_transactions</code>

DML — Работа с данными

Файл	Содержание
<code>insert_data.sql</code>	Примеры INSERT во все 22 таблицы с корректными enum-значениями
<code>update_data.sql</code>	Реалистичные UPDATE: batch collect, SET_ONCE, статусные переходы
<code>select_data.sql</code>	Продвинутые SELECT: DISTINCT ON, рекурсивный CTE, UNION ALL, расчёт виртуального баланса